```
    public string CurrencyFrom { get; set; }
    public string CurrencyTo { get; set; }
    public decimal Quantity { get; set; }
}
```

In FluentValidation you define your validation rules in a separate class, with a class per model to be validated. Typically, these rules derive from the `AbstractValidator<>` base class, which provides a set of extension methods for defining your validation rules.

The following listing shows a validator for the `CurrencyConverterModel`, which matches the validations added using attributes in listing 32.7. You create a set of validation rules for a property by calling `RuleFor()` and chaining method calls such as `NotEmpty()` from it. This style of method chaining is called a *fluent* interface, hence the name.

Listing 32.11 A FluentValidation validator for the currency converter binding model

```
public class CurrencyConverterModelValidator                          ❶
    : AbstractValidator<CurrencyConverterModel>                       ❶
{
    private readonly string[] _allowedValues                          ❷
        = new []{ "GBP", "USD", "CAD", "EUR" };                       ❷

    public CurrencyConverterModelValidator()                          ❸
    {
        RuleFor(x => x.CurrencyFrom)                                  ❹
            .NotEmpty()                                               ❺
            .Length(3)                                                ❺
            .Must(value => _allowedValues.Contains(value))            ❻
            .WithMessage("Not a valid currency code");                ❻

        RuleFor(x => x.CurrencyTo)
            .NotEmpty()
            .Length(3)
            .Must(value => _allowedValues.Contains(value))
            .WithMessage("Not a valid currency code");
```